

ENGLISH FOR WORK / SEPTEMBER 2026

# Software Development English



## Learner workbook

Work with realistic cases, develop your own response, and use the answer section after you have tried the tasks.

### PRACTICE THAT TRANSFERS TO WORK

Read a realistic case. Study complete conversations. Find the right language, speak, get feedback, and try again.

Eight lessons | Intermediate to advanced | Independent or classroom study

For Software developers, application engineers, code reviewers, and maintainers.

All services, packages, versions, measurements, and incidents are fictional language practice, not implementation instructions, cybersecurity guidance, or production operating procedures. Behaviors belong to the expressly described fictional contracts; no language, framework, package manager, or database behavior should be inferred. Check actual documentation, evidence, permissions, and qualified reviewers before technical decisions or production changes.

All scenarios, figures, and model responses are fictional teaching material. Use invented details in practice.

[Open this course on English Ladder](#)

# Your course at a glance

Use the lesson numbers to match this document with the website and the other guides.

## 01 Clarifying interface contracts

Resolve an interface ambiguity using concrete input and outcome pairs.

## 02 Debugging from reproducible evidence

Explain a debugging hypothesis and propose evidence that distinguishes it.

## 03 Giving actionable code-review feedback

Give precise review feedback that separates an acceptable edit from an unintended behavior change.

## 04 Requesting dependency evidence

Ask for evidence that makes a dependency upgrade reviewable.

## 05 Explaining what a test actually exercises

Explain what a test verifies and what its substitutes leave outside scope.

## 06 Comparing performance tradeoffs fairly

Describe a measured benefit and its cost without generalizing beyond the benchmark.

## 07 Discussing retries and shared state

Explain why repeating a state-changing request needs an explicit behavior contract.

## 08 Handing over a release with explicit limits

Give a release handover that distinguishes installed software from enabled behavior and verified recovery.

## Choose your pace

Quick practice: spend 15 minutes reading one conversation and rehearsing one scenario. Full lesson: allow 45-60 minutes for language work, three conversations, two speaking scenarios, and feedback.

## Level support

B1 learners can use the frames and a partner. B2 learners can try a response before reading the model. C1 learners can add the harder follow-up and reduce preparation time. These are teaching suggestions, not certified CEFR level ratings.

After practicing: open the AI prompt workshop, or use the prompts at the end of this guide.

## LESSON 01 / READ AND NOTICE

# Clarifying interface contracts

Resolve an interface ambiguity using concrete input and outcome pairs.

**SITUATION**

A fictional profile interface accepts an optional nickname. The current contract distinguishes an omitted nickname, which leaves it unchanged, from an empty string, which clears it. A proposed client sends null, whose meaning is unspecified.

## 1. Understand the situation

Identify two confirmed facts, one missing detail, and the person who needs your response. Do not fill gaps with invented facts.

## 2. Find the words

interface contract - The specified inputs, outputs, errors, and behavior that callers and an implementation agree to rely on.

nullable field - A data field whose specified allowed values include null, distinct from whether the field may be omitted.

validation error - A reported failure because an input does not meet a stated rule or accepted format.

backward compatibility - The ability of a changed component to continue supporting behavior or inputs relied on by existing consumers.

### Useful word combinations

define the accepted input: state which values an interface allows

distinguish absent from empty: separate an omitted field from a supplied empty value

preserve a caller's expectation: keep behavior an existing consumer relies on

reject an invalid value: return a specified failure for input outside the contract

## LESSON 01 / LANGUAGE AND PRACTICE

# Ask about one ambiguous input

## 3. Notice the language

Explain the specified cases before asking about the gap. 'Optional' concerns whether a field must be present; 'nullable' concerns whether null is accepted. Neither automatically tells a caller what an empty string does.

For input \_\_\_, the contract specifies \_\_\_.

Does \_\_\_ mean \_\_\_, or should it be rejected?

That would change the behavior expected by \_\_\_.

Compare this wording: Optional means you can send anything blank.

Clearer: Omission leaves the nickname unchanged; an empty string clears it. Null still needs a defined meaning.

The revision distinguishes three inputs instead of grouping them under the ambiguous word blank.

## 4. Check the language

1. Client developer: Can I send null to clear the nickname?

Service developer: The contract defines an empty string for clearing; null is \_\_\_.

- A. deprecated
- B. unspecified
- C. equivalent
- D. required

2. Reviewer: Does optional mean nullable?

Developer: No. Optional concerns presence, \_\_\_ nullable concerns accepting null.

- A. whereas
- B. therefore
- C. because
- D. unless

## 05 • Conversations

The next three pages contain original fictional conversations for this lesson. Each numbered line is one speaking turn. Read the setting before assigning roles.

## LESSON 01 / 05 · CONVERSATIONS / 1 OF 3

# What does null do to a nickname?

Two fictional developers resolve an unspecified input in the profile contract.

**01 · Client developer:** I want to clear a nickname. Can my client send null for that?

**02 · Service developer:** The contract specifies an empty string for clearing. Null doesn't currently have a defined meaning.

**03 · Client developer:** How is an optional field different from a nullable field in this interface contract?

**04 · Service developer:** Optional allows omission; nullable allows null. Here, omission leaves the nickname unchanged; an empty string clears it.

**05 · Client developer:** Then null shouldn't silently mean unchanged on one client and cleared on another.

**06 · Service developer:** Agreed. Shall we propose rejecting null with a validation error until another meaning is agreed?

**07 · Client developer:** That keeps the current clearing behavior explicit. My client can send an empty string.

**08 · Service developer:** Please also preserve omission for updates that don't intend to change the nickname.

**09 · Client developer:** I will. Those are two different user intentions, even if both look blank.

**10 · Service developer:** Then our proposed contract keeps both outcomes and identifies null as unsupported, pending review.

## Notice, then rehearse

Name each speaker's purpose. Identify one useful expression and the next step or unresolved question. Read the roles aloud, then switch roles.

## LESSON 01 / 05 • CONVERSATIONS / 2 OF 3

# An empty list is a successful result

Fictional developers clarify the difference between no search matches and a failed search.

**01 • Consumer developer:** Our search adapter treats an empty list as an error. Is that intended?

**02 • Interface owner:** No. In this contract, an empty list means the search succeeded without finding matches.

**03 • Consumer developer:** What represents a failed request? I need to show a different message for that.

**04 • Interface owner:** A validation error has a separate error result. It doesn't use the successful list shape.

**05 • Consumer developer:** So no matches should display an empty state, while invalid input should explain the error?

**06 • Interface owner:** Exactly. The interface contract separates those outcomes even though neither produces visible result rows.

**07 • Consumer developer:** I'll remove the assumption that a successful result must contain at least one item.

**08 • Interface owner:** Please add the zero-match example beside the populated example when reviewing the adapter.

**09 • Consumer developer:** That will make the distinction concrete. We can check both without guessing from list length.

**10 • Interface owner:** And callers can offer another search without telling users that a successful request failed.

## Notice, then rehearse

Name each speaker's purpose. Identify one useful expression and the next step or unresolved question. Read the roles aloud, then switch roles.

## LESSON 01 / 05 • CONVERSATIONS / 3 OF 3

# A renamed response field breaks an older caller

A fictional code review identifies a compatibility concern in a response change.

**01 • Reviewer:** The new response replaces `displayName` with `label`. What happens to the older caller?

**02 • Developer:** It still reads `displayName`. I changed the server response but haven't changed that client.

**03 • Reviewer:** Then this affects backward compatibility, not simply an internal variable name.

**04 • Developer:** You're right. The field crosses the interface boundary, so consumers can observe the difference.

**05 • Reviewer:** Do we know whether both fields could temporarily coexist under an agreed transition contract?

**06 • Developer:** That needs discussion with the client owner. We haven't specified precedence if values differ.

**07 • Reviewer:** Please keep the response change out of the internal cleanup until that contract is settled.

**08 • Developer:** I can separate it. The local variable rename doesn't require changing the response field.

**09 • Reviewer:** That gives us a focused review and keeps the existing caller's expectation intact.

**10 • Developer:** I'll retain `displayName` in this change and open the proposed interface transition for discussion.

## Notice, then rehearse

Name each speaker's purpose. Identify one useful expression and the next step or unresolved question. Read the roles aloud, then switch roles.

## LESSON 01 / 06 • SAY IT

## Two scenarios to practice

Use only the supplied facts. Prepare for two minutes, speak for one minute, then respond to a follow-up. Switch roles and repeat. For solo study, speak both roles aloud.

### Scenario 1 | Clarifying interface contracts

A fictional profile interface accepts an optional nickname. The current contract distinguishes an omitted nickname, which leaves it unchanged, from an empty string, which clears it. A proposed client sends null, whose meaning is unspecified.

Prepare for two minutes. Speak for 45-60 seconds using the case facts, then respond to your partner's question. Switch roles and repeat without reading the model response.

Partner: Client developer: you want one way to clear a nickname, and your current client can send either an empty string or null.

Second round: A colleague calls omission and an empty string interchangeable. Restate their different outcomes in this fictional contract.

### Scenario 2 | A required currency without a default

A fictional pricing interface requires a currency code. A client omits it, assuming dollars. The contract supplies no default. The interface owner can confirm that omission is invalid; no conversion is involved.

Role A: Client developer: ask whether omission has a supported meaning and restate the answer.

Role B: Interface owner: explain that currency is required and a value must be supplied explicitly.

Possible opening: Does an omitted currency mean dollars in this contract, or is it invalid?

### Success checks

- Separate example data from the contract.
- Confirm currency must be explicit.
- Avoid inventing a conversion or default behavior.

Add a complication: Someone points to a dollar sign in an example and calls it a default rule.

Check: facts accurate | meaning clear | tone appropriate | next action or question explicit

Continue scenario 1 with an AI partner.

## LESSON 02 / READ AND NOTICE

# Debugging from reproducible evidence

Explain a debugging hypothesis and propose evidence that distinguishes it.

**SITUATION**

A fictional import fails on a three-row file containing an empty date field. A two-row file without that row succeeds. The stack trace points to date parsing, but no controlled test has isolated the empty field as the cause.

## 1. Understand the situation

Identify two confirmed facts, one missing detail, and the person who needs your response. Do not fill gaps with invented facts.

## 2. Find the words

reproduction - A documented set of conditions and actions that produces an observed software behavior again.

stack trace - A record of function calls associated with a point in execution, often captured when an error occurs.

hypothesis - A proposed explanation for observed behavior that can be investigated against evidence.

minimal example - A reduced input or program that retains the behavior being investigated while removing irrelevant detail.

**Useful word combinations**

reproduce a failure: make the reported behavior occur under described conditions

narrow the failing input: remove irrelevant detail while retaining the failure

inspect the call path: examine which operations led to the observed failure

rule out a hypothesis: use evidence to reject a proposed explanation

## LESSON 02 / LANGUAGE AND PRACTICE

# Separate a clue from a cause

## 3. Notice the language

A stack trace shows a path associated with failure, not necessarily the original cause. When two examples differ in several ways, identify the confounding differences before claiming that one field explains the failure.

The failure occurs with \_\_\_, but \_\_\_ also differs.

The trace points to \_\_\_; that does not yet establish \_\_\_.

A comparison holding \_\_\_ constant would help distinguish \_\_\_.

Compare this wording: The parser is broken because the file has an empty date.

Clearer: The failure reaches date parsing; the empty date is a hypothesis that still needs an isolated comparison.

The revised claim matches the evidence and names the untested explanation.

## 4. Check the language

1. Colleague: Does the passing two-row file prove the empty date caused the failure?

Developer: No; it also changes the \_\_\_, so that comparison does not isolate the date.

- A. row count
- B. date format
- C. delimiter
- D. character encoding

2. Reviewer: How certain is your explanation?

Developer: The trace \_\_\_ a parsing issue, but we haven't isolated the cause.

- A. rules out
- B. guarantees
- C. is unrelated to
- D. suggests

## 05 • Conversations

The next three pages contain original fictional conversations for this lesson. Each numbered line is one speaking turn. Read the setting before assigning roles.

## LESSON 02 / 05 • CONVERSATIONS / 1 OF 3

# The empty date is still a hypothesis

Fictional developers discuss a failing three-row import and a passing two-row file.

**01 • Developer:** The three-row import fails in date parsing. I suspect the row with an empty date.

**02 • Colleague:** Does your passing example differ only in that date, or in other ways?

**03 • Developer:** It has two rows instead of three, so the comparison also changes row count.

**04 • Colleague:** Then it doesn't isolate the empty field. What does the stack trace actually show?

**05 • Developer:** The call path reaches date parsing before the failure; it doesn't establish the original cause.

**06 • Colleague:** Could we keep three rows and change only the disputed date in a controlled example?

**07 • Developer:** Yes. That would distinguish the date hypothesis from the row-count difference in these files.

**08 • Colleague:** Please preserve the failing input too, so another developer can reproduce the observation.

**09 • Developer:** I've labeled the cause unconfirmed and attached both existing examples with their differences.

**10 • Colleague:** Good. We have a specific hypothesis and a comparison that can test it.

## Notice, then rehearse

Name each speaker's purpose. Identify one useful expression and the next step or unresolved question. Read the roles aloud, then switch roles.

## LESSON 02 / 05 • CONVERSATIONS / 2 OF 3

# The error line is not necessarily where the defect began

Fictional developers inspect a trace from a report formatter.

**01 • Developer:** The trace points to the formatter. Should we assign the whole defect there?

**02 • Reviewer:** Which value reached it? A failure location doesn't tell us where invalid state originated.

**03 • Developer:** The formatter expected a title, but the input object contains no title field.

**04 • Reviewer:** Does the upstream contract require that field, or may callers legitimately omit it?

**05 • Developer:** It requires a title. The minimal example bypasses the normal builder and leaves it out.

**06 • Reviewer:** Then we've identified a contract violation in the example, not yet a formatter implementation defect.

**07 • Developer:** That's useful. The user report may still involve a missing title from the real builder.

**08 • Reviewer:** Exactly. We need the actual path before deciding whether the example represents that report.

**09 • Developer:** I'll separate the synthetic missing-field failure from the unconfirmed production report in the discussion.

**10 • Reviewer:** And I'll review the builder's output contract with you rather than blame the final trace line.

## Notice, then rehearse

Name each speaker's purpose. Identify one useful expression and the next step or unresolved question. Read the roles aloud, then switch roles.

## LESSON 02 / 05 • CONVERSATIONS / 3 OF 3

# One successful rerun doesn't establish a correction

Fictional developers discuss intermittent behavior without claiming a fix.

**01 • Maintainer:** The import passed on its second attempt. Can we close the failure report?

**02 • Developer:** No code changed between attempts. A passing rerun doesn't establish that the cause was removed.

**03 • Maintainer:** Were the input and build the same, or did something change without being noted?

**04 • Developer:** The file and build match. We didn't capture the external response from the first attempt.

**05 • Maintainer:** So an external difference is possible, but we don't have evidence identifying one.

**06 • Developer:** Right. I'll record one failure and one success under those known matching conditions.

**07 • Maintainer:** Can you retain the first trace instead of replacing it with the successful output?

**08 • Developer:** Yes. Both outcomes matter, especially because we cannot yet explain their difference.

**09 • Maintainer:** Then the status is intermittent observation, cause open, and no verified correction.

**10 • Developer:** Agreed. The next reproduction needs the missing response context, not just another success count.

## Notice, then rehearse

Name each speaker's purpose. Identify one useful expression and the next step or unresolved question. Read the roles aloud, then switch roles.

## LESSON 02 / 06 • SAY IT

## Two scenarios to practice

Use only the supplied facts. Prepare for two minutes, speak for one minute, then respond to a follow-up. Switch roles and repeat. For solo study, speak both roles aloud.

### Scenario 1 | Debugging from reproducible evidence

A fictional import fails on a three-row file containing an empty date field. A two-row file without that row succeeds. The stack trace points to date parsing, but no controlled test has isolated the empty field as the cause.

Prepare for two minutes. Speak for 45-60 seconds using the case facts, then respond to your partner's question. Switch roles and repeat without reading the model response.

Partner: Developer receiving the bug: ask which inputs and conditions differ between the passing and failing examples.

Second round: The same failing file succeeds once after a retry. Explain why one success does not establish a fix.

### Scenario 2 | A failure that depends on input order

In a fictional example, records A then B produce a duplicate label, while B then A do not. The same records and build are used. No internal cause has been established.

Role A: Developer: describe the observed ordering difference and propose a controlled comparison.

Role B: Maintainer: ask what is identical and whether the result proves a particular internal defect.

Possible opening: The same two records produce different results when their order changes.

### Success checks

- Preserve the order-sensitive observation.
- Keep the internal explanation unconfirmed.
- Agree a smaller order comparison with recorded outputs.

Add a complication: A colleague declares the data corrupted even though the records are unchanged.

Check: facts accurate | meaning clear | tone appropriate | next action or question explicit

Continue scenario 1 with an AI partner.

## LESSON 03 / READ AND NOTICE

# Giving actionable code-review feedback

Give precise review feedback that separates an acceptable edit from an unintended behavior change.

**SITUATION**

A fictional pull request is titled 'rename export helper.' Its diff also moves a notification call outside a success branch, so failed exports would now notify the user as if complete. The naming change is acceptable; the behavior change is unintended.

## 1. Understand the situation

Identify two confirmed facts, one missing detail, and the person who needs your response. Do not fill gaps with invented facts.

## 2. Find the words

pull request - A proposal to incorporate a set of code changes, commonly accompanied by a diff and review discussion.

diff - A representation showing added, removed, or changed content between two versions.

refactoring - Restructuring existing code while preserving its observable behavior.

side effect - An observable effect beyond a function's returned value, such as changing stored state or sending a notification.

### Useful word combinations

keep a change focused: limit a review to a coherent behavior or purpose

trace a side effect: follow a change outside a function's returned value

separate a preference from a blocker: distinguish optional style from required correction

preserve existing behavior: keep observable outcomes unchanged during restructuring

## LESSON 03 / LANGUAGE AND PRACTICE

# Comment on behavior and impact

## 3. Notice the language

Point to the observed change, explain the resulting behavior, and request a bounded revision. Refactoring normally preserves observable behavior; the label on a pull request does not guarantee that its diff does so.

The change moves \_\_\_, which means \_\_\_.

My concern is \_\_\_, not \_\_\_.

Could you preserve \_\_\_ and add evidence for \_\_\_?

Compare this wording: This refactor is bad.

Clearer: Moving the notification outside the success branch changes the failure path; please keep it conditional on success.

The review now names the defect and a concrete revision rather than judging the author or the entire change.

## 4. Check the language

1. Author: Is the helper name the blocker?

Reviewer: No. The blocker is the changed \_\_\_ on failed exports.

- A. return value
- B. accepted input
- C. helper signature
- D. side effect

2. Author: Should I redo the whole pull request?

Reviewer: The rename can stay; please \_\_\_ the notification's success condition.

- A. announce
- B. summarize
- C. preserve
- D. infer

## 05 • Conversations

The next three pages contain original fictional conversations for this lesson. Each numbered line is one speaking turn. Read the setting before assigning roles.

## LESSON 03 / 05 • CONVERSATIONS / 1 OF 3

# A rename that also moves a notification

A fictional author and reviewer resolve an unintended side effect in a pull request.

**01 • Reviewer:** The helper rename is clear, but moving the notification changes the failed-export path.

**02 • Author:** I intended only a rename. Which movement changes what the user would see?

**03 • Reviewer:** The call now sits outside the success branch, so failure would also produce a completion notification.

**04 • Author:** You're right. That's an unintended side effect, not part of the requested cleanup.

**05 • Reviewer:** Could you retain the success condition and add a check for the failure path?

**06 • Author:** Yes. I'll restore that condition and assert that failed exports don't show completion.

**07 • Reviewer:** The naming choice isn't a blocker. My required change concerns the notification's behavior.

**08 • Author:** That distinction helps. I'll leave the name and focus the revision on the conditional call.

**09 • Reviewer:** Once revised, I'll review the changed branch and the new assertion together.

**10 • Author:** I've accepted that request. This remains a rename with preserved behavior, not a notification redesign.

## Notice, then rehearse

Name each speaker's purpose. Identify one useful expression and the next step or unresolved question. Read the roles aloud, then switch roles.

## LESSON 03 / 05 • CONVERSATIONS / 2 OF 3

# An optional suggestion among required changes

Fictional reviewers distinguish a readability preference from an incorrect boundary check.

**01 • Author:** You left comments about the variable name and the upper-bound check. Are both blockers?

**02 • Reviewer:** The boundary check is required: this contract accepts ten, but the new comparison rejects it.

**03 • Author:** And the name suggestion? I used the same name as the neighboring helper.

**04 • Reviewer:** That's optional. I find limit clearer, but the existing naming convention is defensible.

**05 • Author:** I'll correct the boundary and keep the established name to avoid unrelated changes.

**06 • Reviewer:** That works. Please include the accepted value ten and the rejected value eleven in the evidence.

**07 • Author:** Those examples make the intended boundary clear without expanding this into a style cleanup.

**08 • Reviewer:** Exactly. My review should distinguish a behavior mismatch from a preference about readability.

**09 • Author:** I'll mark the naming thread as discussed and return the boundary revision for review.

**10 • Reviewer:** Then the required action is unambiguous, and the optional comment doesn't become an accidental condition.

## Notice, then rehearse

Name each speaker's purpose. Identify one useful expression and the next step or unresolved question. Read the roles aloud, then switch roles.

## LESSON 03 / 05 • CONVERSATIONS / 3 OF 3

# A helper extracts more behavior than expected

Fictional developers discuss a refactor that unexpectedly reorders an observable event.

**01 • Reviewer:** The extracted helper sends the receipt before the save completes. Was that ordering change intentional?

**02 • Author:** No. I moved both operations into one helper but changed their order accidentally.

**03 • Reviewer:** The existing behavior sends only after a successful save. Callers may rely on that distinction.

**04 • Author:** Refactoring should preserve that observable sequence, not merely reuse the same calls.

**05 • Reviewer:** Could you keep the original ordering and explain the helper's responsibility more narrowly?

**06 • Author:** Yes. It will coordinate the existing sequence rather than introduce a new notification policy.

**07 • Reviewer:** Please keep the unrelated retry proposal outside this pull request so we can review each behavior separately.

**08 • Author:** I'll separate that proposal. The current revision will only extract the unchanged sequence.

**09 • Reviewer:** That gives the tests a clear expectation: no receipt before the successful save outcome.

**10 • Author:** And it makes the refactor claim something we can actually check against observable behavior.

## Notice, then rehearse

Name each speaker's purpose. Identify one useful expression and the next step or unresolved question. Read the roles aloud, then switch roles.

## LESSON 03 / 06 • SAY IT

## Two scenarios to practice

Use only the supplied facts. Prepare for two minutes, speak for one minute, then respond to a follow-up. Switch roles and repeat. For solo study, speak both roles aloud.

### Scenario 1 | Giving actionable code-review feedback

A fictional pull request is titled 'rename export helper.' Its diff also moves a notification call outside a success branch, so failed exports would now notify the user as if complete. The naming change is acceptable; the behavior change is unintended.

Prepare for two minutes. Speak for 45-60 seconds using the case facts, then respond to your partner's question. Switch roles and repeat without reading the model response.

Partner: Author: the rename is intentional, but moving the notification was accidental. Ask which path needs a check.

Second round: A second reviewer adds an optional naming preference. Label it optional while keeping the failure-path correction clearly required.

### Scenario 2 | A review comment that needs an example

A fictional review says 'This loses precision.' The author rounded a displayed duration, but the underlying calculation remains unchanged. The reviewer is concerned only about the displayed boundary between 0.9 and 1.0 seconds.

Role A: Author: ask which value, output, and consequence the comment refers to.

Role B: Reviewer: give the 0.9-to-1.0 display example and distinguish it from calculation accuracy.

Possible opening: Could you identify which output loses precision and whether your concern is display or calculation?

### Success checks

- Ask for the concrete example.
- Separate presentation from underlying computation.
- Agree a focused review of display rounding.

Add a complication: A teammate proposes rewriting the calculation before the display concern is clarified.

Check: facts accurate | meaning clear | tone appropriate | next action or question explicit

Continue scenario 1 with an AI partner.

## LESSON 04 / READ AND NOTICE

# Requesting dependency evidence

Ask for evidence that makes a dependency upgrade reviewable.

**SITUATION**

A fictional application directly uses package ParcelKit, which brings in TextCore indirectly. A proposed ParcelKit upgrade changes the resolved TextCore version. The change note mentions a deprecated parsing entry point; the team's usage and replacement compatibility are unknown.

## 1. Understand the situation

Identify two confirmed facts, one missing detail, and the person who needs your response. Do not fill gaps with invented facts.

## 2. Find the words

direct dependency - A component explicitly required by the application or package being discussed.

transitive dependency - A component required indirectly through another dependency in the dependency chain.

lockfile - A tool-specific record of resolved dependencies used to help reproduce a selected dependency set.

deprecation - A notice that an interface or feature is discouraged for future use, often ahead of possible removal.

**Useful word combinations**

trace the dependency path: identify which package brings another package into a build

review an upgrade note: examine documented changes before adopting a version

pin a resolved version: record a specific selected version where the tool supports it

check the supported interface: verify which calls remain available to consumers

## LESSON 04 / LANGUAGE AND PRACTICE

# Request the path, version, and affected call

## 3. Notice the language

Direct and transitive describe how a dependency enters the application. Deprecation warns that an interface should be replaced or may be removed; it does not by itself establish removal now. Tool and package behavior must be checked in their actual documentation.

Which dependency introduces \_\_\_?

Could you link the change note and identify our use of \_\_\_?

Does the resolved version change from \_\_\_ to \_\_\_?

Compare this wording: It's a small version change, so it won't affect us.

Clearer: Please show the resolved dependency change and check whether our callers use the deprecated entry point.

The revision requests evidence rather than treating a version label as a compatibility guarantee.

## 4. Check the language

1. Reviewer: We don't list TextCore directly. How does it enter the build?

Developer: For our application, TextCore is a \_\_\_ dependency introduced by ParcelKit.

- A. optional
- B. development-only
- C. transitive
- D. direct

2. Developer: The note says deprecated. Does that prove the entry point is absent now?

Reviewer: No; deprecation and \_\_\_ are different states.

- A. resolution
- B. removal
- C. documentation
- D. installation

## 05 • Conversations

The next three pages contain original fictional conversations for this lesson. Each numbered line is one speaking turn. Read the setting before assigning roles.

## LESSON 04 / 05 • CONVERSATIONS / 1 OF 3

# The package we didn't list directly

Fictional developers identify evidence needed for a ParcelKit upgrade review.

**01 • Reviewer:** ParcelKit is our direct dependency; what introduces TextCore into the proposed build?

**02 • Maintainer:** TextCore is a transitive dependency through ParcelKit. Its resolved version changes in the proposed lockfile.

**03 • Reviewer:** Could you show that difference and link the note about the deprecated parsing entry point?

**04 • Maintainer:** I have both. I haven't checked whether our application or ParcelKit uses that entry point.

**05 • Reviewer:** Please identify the caller before describing this as an update requiring no code changes.

**06 • Maintainer:** Agreed. Deprecation doesn't automatically mean removal, but the actual call and version still matter.

**07 • Reviewer:** Exactly. We need evidence about this combination, not an assumption based on the package name.

**08 • Maintainer:** I'll return the dependency path, resolved versions, change note, and relevant call-site findings.

**09 • Reviewer:** That will let us separate package resolution from compatibility with the calls we make.

**10 • Maintainer:** And we'll leave the drop-in claim unconfirmed until those findings support it.

## Notice, then rehearse

Name each speaker's purpose. Identify one useful expression and the next step or unresolved question. Read the roles aloud, then switch roles.

## LESSON 04 / 05 • CONVERSATIONS / 2 OF 3

# A lockfile difference without a source change

Fictional maintainers discuss why two builds resolved different dependency sets.

**01 • Developer:** My source changes only a label, yet the proposed build resolves three different package versions.

**02 • Maintainer:** Did the dependency manifest change, or only the lockfile that records this resolution?

**03 • Developer:** Only the lockfile differs. I don't know which tool action regenerated it locally.

**04 • Maintainer:** Then the label change and dependency change need separate explanations before we call the build equivalent.

**05 • Developer:** Could I describe the lockfile as generated and therefore irrelevant to behavior?

**06 • Maintainer:** No. Generated doesn't mean inconsequential; the resolved dependencies may affect what goes into the artifact.

**07 • Developer:** I'll identify how it changed and compare the selected versions with the current build.

**08 • Maintainer:** Please check the actual tool documentation rather than assume every package manager treats lockfiles identically.

**09 • Developer:** Understood. The source edit remains narrow, but the artifact difference is not yet explained.

**10 • Maintainer:** That's the right scope for review: one known label change and an open resolution difference.

## Notice, then rehearse

Name each speaker's purpose. Identify one useful expression and the next step or unresolved question. Read the roles aloud, then switch roles.

## LESSON 04 / 05 • CONVERSATIONS / 3 OF 3

# Deprecated is not removed

A fictional maintainer clarifies an upgrade note with a caller's developer.

**01 • Caller developer:** The note says the old reader is deprecated. Does our current call stop working immediately?

**02 • Maintainer:** The note doesn't establish removal. It says the replacement is preferred, with removal timing unspecified.

**03 • Caller developer:** So we should distinguish a migration recommendation from a confirmed missing entry point.

**04 • Maintainer:** Yes. We also need to compare the replacement's input and error behavior before changing callers.

**05 • Caller developer:** Our current caller expects an error result for malformed input. Does the replacement preserve that?

**06 • Maintainer:** The note links a contract section, but I haven't checked that specific behavior yet.

**07 • Caller developer:** Please include it in the review. A matching function name wouldn't settle that difference.

**08 • Maintainer:** I'll compare the stated contracts and identify any behavior our caller relies on.

**09 • Caller developer:** Then today's conclusion is deprecated use identified, replacement compatibility still under review.

**10 • Maintainer:** Exactly. We can discuss a migration without inventing an immediate failure or a removal deadline.

## Notice, then rehearse

Name each speaker's purpose. Identify one useful expression and the next step or unresolved question. Read the roles aloud, then switch roles.

## LESSON 04 / 06 • SAY IT

## Two scenarios to practice

Use only the supplied facts. Prepare for two minutes, speak for one minute, then respond to a follow-up. Switch roles and repeat. For solo study, speak both roles aloud.

### Scenario 1 | Requesting dependency evidence

A fictional application directly uses package ParcelKit, which brings in TextCore indirectly. A proposed ParcelKit upgrade changes the resolved TextCore version. The change note mentions a deprecated parsing entry point; the team's usage and replacement compatibility are unknown.

Prepare for two minutes. Speak for 45-60 seconds using the case facts, then respond to your partner's question. Switch roles and repeat without reading the model response.

Partner: Maintainer: you have the change note and resolved dependency list, but have not searched the application's call sites yet.

Second round: The package name is unchanged. Explain why that alone does not establish identical behavior across versions.

### Scenario 2 | Two components request different versions

In a fictional dependency report, App requests FormatKit 1.4; App -> ViewPlug requests FormatKit 2.1. The supplied graph retains both versions at those paths; the tool's resolution rules remain unreviewed.

Role A: Developer: ask what was actually resolved and where each request originates.

Role B: Maintainer: show the two requests and state that the tool's resolution rules need checking.

Possible opening: These are two requested versions; which versions actually appear in the resolved graph?

#### Success checks

- Distinguish requested from resolved versions.
- Identify both dependency paths.
- Avoid claiming a universal resolution rule.

Add a complication: Someone assumes the higher version always wins in every package tool.

Check: facts accurate | meaning clear | tone appropriate | next action or question explicit

Continue scenario 1 with an AI partner.

## LESSON 05 / READ AND NOTICE

# Explaining what a test actually exercises

Explain what a test verifies and what its substitutes leave outside scope.

**SITUATION**

A fictional save function's unit test replaces storage with a test double that always reports success. It checks the success message but never exercises a storage rejection. A previous bug incorrectly showed success after rejection; the real storage adapter is not tested here.

## 1. Understand the situation

Identify two confirmed facts, one missing detail, and the person who needs your response. Do not fill gaps with invented facts.

## 2. Find the words

unit test - A test focused on a small software unit, often isolating it from external collaborators.

integration test - A test examining interactions between combined components or with an external component.

test double - A controlled substitute for a real collaborator used during a test.

regression test - A test intended to detect unintended changes in previously working or corrected behavior.

**Useful word combinations**

exercise an error path: run the behavior used when an operation fails

replace a collaborator with a test double: use a controlled substitute during a test

assert an observable result: check an outcome rather than only execution

add a regression case: preserve a check for a previously observed defect

## LESSON 05 / LANGUAGE AND PRACTICE

# Describe the boundary of a test

## 3. Notice the language

Name the real component, the replaced collaborator, and the assertion. A test can run quickly and still omit the behavior in question. A passing result supports its actual checks, not every possible interaction.

This test exercises \_\_\_ while replacing \_\_\_.

It asserts \_\_\_, but it does not cover \_\_\_.

A separate test is needed to examine \_\_\_.

Compare this wording: The save feature is tested.

Clearer: The unit test checks a successful save response using simulated storage; rejection and the real adapter are not covered.

The revision identifies the checked behavior and the boundaries of the evidence.

## 4. Check the language

1. Reviewer: Why didn't this test catch the rejection bug?

Developer: Its storage substitute \_\_\_\_.

- A. checks every rejection
- B. always reports success
- C. returns both outcomes
- D. uses the real database

2. Developer: The new case repeats the earlier failure and checks the corrected message.

Reviewer: That makes it a \_\_\_ test for that bug.

- A. regression
- B. load
- C. usability
- D. migration

## 05 • Conversations

The next three pages contain original fictional conversations for this lesson. Each numbered line is one speaking turn. Read the setting before assigning roles.

## LESSON 05 / 05 • CONVERSATIONS / 1 OF 3

# The storage substitute never rejects

Fictional developers examine why a unit test missed a misleading success message.

**01 • Reviewer:** The save test passed before the bug report. What outcome did its storage substitute return?

**02 • Developer:** It always returned success, and the assertion checked that the success message appeared.

**03 • Reviewer:** Then it never exercised the rejection that previously produced the wrong message, did it?

**04 • Developer:** No. We need a regression case where the substitute rejects and the visible result is checked.

**05 • Reviewer:** Would asserting only that the function returns without throwing catch the misleading success message?

**06 • Developer:** Not necessarily. The assertion needs to distinguish the intended rejection message from false success.

**07 • Reviewer:** Good. And what would that case tell us about the real storage adapter?

**08 • Developer:** Only about the save function's response to the simulated outcome, not the adapter's actual behavior.

**09 • Reviewer:** Then name the integration question separately so the unit test isn't credited with answering it.

**10 • Developer:** The unit check covers rejection response; an integration test must examine the real adapter.

## Notice, then rehearse

Name each speaker's purpose. Identify one useful expression and the next step or unresolved question. Read the roles aloud, then switch roles.

## LESSON 05 / 05 • CONVERSATIONS / 2 OF 3

# A test that calls code but checks no result

Fictional developers strengthen the stated purpose of a parsing test.

**01 • Developer:** This new test calls the parser with a valid record and completes without an exception.

**02 • Reviewer:** What behavior is it intended to establish? Execution alone doesn't check the parsed values.

**03 • Developer:** It should preserve leading zeros in a reference, but I haven't asserted that result.

**04 • Reviewer:** Then a parser returning a different reference might pass this test as currently described.

**05 • Developer:** You're right. The concrete input is zero zero seven, and the expected reference retains all three characters.

**06 • Reviewer:** Please make that output the check, rather than relying on successful execution as a substitute.

**07 • Developer:** That also explains why converting the reference to a number would violate this contract.

**08 • Reviewer:** Exactly. The test's purpose is preserving an identifier, not checking arithmetic on its digits.

**09 • Developer:** I'll add the expected reference and name the case around preserving leading zeros.

**10 • Reviewer:** Then a failure will point to the behavior we care about instead of an unspecified parser problem.

## Notice, then rehearse

Name each speaker's purpose. Identify one useful expression and the next step or unresolved question. Read the roles aloud, then switch roles.

## LESSON 05 / 05 • CONVERSATIONS / 3 OF 3

# A test double with the wrong contract

Fictional developers find a mismatch between a controlled substitute and a real collaborator.

**01 • Reviewer:** The test double returns a plain string, but the collaborator's contract returns a result object.

**02 • Developer:** I simplified it because the function only needs the label inside that object.

**03 • Reviewer:** Does the function actually extract the label, or does the simplified substitute hide that step?

**04 • Developer:** It extracts the label in production. The test currently bypasses that shape difference.

**05 • Reviewer:** Then the substitute is not exercising the interface the function is supposed to consume.

**06 • Developer:** I'll make its returned shape match the contract and keep the sample label simple.

**07 • Reviewer:** Please include the error result shape too if this case claims to cover rejection.

**08 • Developer:** This case covers success only. I'll label it narrowly and add rejection as a separate case.

**09 • Reviewer:** That separates test data simplicity from changing the collaborator's promised interface.

**10 • Developer:** Agreed. A controlled substitute can be predictable without teaching the test the wrong contract.

## Notice, then rehearse

Name each speaker's purpose. Identify one useful expression and the next step or unresolved question. Read the roles aloud, then switch roles.

## LESSON 05 / 06 • SAY IT

## Two scenarios to practice

Use only the supplied facts. Prepare for two minutes, speak for one minute, then respond to a follow-up. Switch roles and repeat. For solo study, speak both roles aloud.

### Scenario 1 | Explaining what a test actually exercises

A fictional save function's unit test replaces storage with a test double that always reports success. It checks the success message but never exercises a storage rejection. A previous bug incorrectly showed success after rejection; the real storage adapter is not tested here.

Prepare for two minutes. Speak for 45-60 seconds using the case facts, then respond to your partner's question. Switch roles and repeat without reading the model response.

Partner: Reviewer: ask which collaborator is substituted and what assertion would fail if the success message reappeared on rejection.

Second round: A teammate proposes checking only that no exception was thrown. Explain why the visible message needs its own assertion.

### Scenario 2 | A test name overstates its assertion

A fictional test named 'saves and reloads settings' checks only the save response. It uses a test double and never performs a reload. The author can rename it and propose a distinct persistence check.

Role A: Reviewer: explain the mismatch between the test name and observed actions.

Role B: Author: distinguish a response check from a read-after-save interaction.

Possible opening: This test checks the save response, but where does it reload the settings?

#### Success checks

- State exactly what is asserted.
- Avoid treating a response as proof of persistence.
- Agree a truthful name and a separate interaction question.

Add a complication: A teammate says the positive response makes the reload check redundant.

Check: facts accurate | meaning clear | tone appropriate | next action or question explicit

Continue scenario 1 with an AI partner.

## LESSON 06 / READ AND NOTICE

# Comparing performance tradeoffs fairly

Describe a measured benefit and its cost without generalizing beyond the benchmark.

**SITUATION**

In a fictional controlled benchmark on the same input and machine, implementation A has median latency 40 milliseconds and memory use 80 megabytes; B has 25 milliseconds and 160 megabytes. The available memory budget is 120 megabytes. Tail latency and production workloads are unmeasured.

## 1. Understand the situation

Identify two confirmed facts, one missing detail, and the person who needs your response. Do not fill gaps with invented facts.

## 2. Find the words

latency - The elapsed time between a specified request or action and its measured response or completion.

throughput - The amount of work completed per unit of time under specified conditions.

benchmark - A defined measurement exercise used to compare performance under stated workloads and conditions.

memory footprint - The amount of memory used by a program or component under specified conditions.

### Useful word combinations

hold the workload constant: compare implementations with equivalent inputs and conditions

report a distribution: show variation rather than only a single average

trade memory for speed: use more memory to reduce a measured delay

separate warm and cold runs: distinguish runs with and without relevant initialized state

## LESSON 06 / LANGUAGE AND PRACTICE

# Compare the same measurements and constraints

## 3. Notice the language

Latency measures delay; throughput measures work completed over time. A median is not a maximum, and a result for one workload is not a universal prediction. Use the same units and the stated resource constraint.

Under the same \_\_\_, A measured \_\_\_ whereas B measured \_\_\_.

The improvement in \_\_\_ comes with \_\_\_.

We have not measured \_\_\_, so that claim remains open.

Compare this wording: B is faster, so we should use it.

Clearer: B lowers median latency but exceeds our memory budget; A stays within it. Tail latency is still unmeasured.

The revised comparison includes the limiting resource and the boundary of the measurements.

## 4. Check the language

1. Reviewer: Which implementation fits the stated memory budget?

Developer: \_\_\_ uses eighty megabytes, below the one-hundred-twenty-megabyte limit.

A. Implementation A

B. Both implementations

C. Neither implementation

D. Implementation B

2. Colleague: Does the median tell us the longest measured wait?

Developer: No; to report the longest observed wait, we need the measured \_\_\_.

A. mean

B. minimum

C. median

D. maximum

## 05 • Conversations

The next three pages contain original fictional conversations for this lesson. Each numbered line is one speaking turn. Read the setting before assigning roles.

## LESSON 06 / 05 • CONVERSATIONS / 1 OF 3

# Faster at the median, outside the memory budget

Fictional developers compare implementations A and B under the supplied benchmark.

**01 • Developer:** B's median latency is twenty-five milliseconds, versus forty for A. Should we choose B?

**02 • Reviewer:** How does each memory footprint compare with our stated budget?

**03 • Developer:** A uses eighty megabytes; B uses one hundred sixty. Our budget is one hundred twenty.

**04 • Reviewer:** Then B improves that latency measure but exceeds the resource constraint we're working within.

**05 • Developer:** Agreed. A fits the memory budget, though its median is fifteen milliseconds slower.

**06 • Reviewer:** Do these results tell us about the slowest requests or only the median?

**07 • Developer:** Only the median. We haven't measured tail latency or representative production workloads yet.

**08 • Reviewer:** Please include those limits instead of describing B as universally faster and therefore better.

**09 • Developer:** I'll present A as within budget and B as requiring a memory tradeoff we haven't accepted.

**10 • Reviewer:** That comparison gives us a decision constraint, not just the most attractive single number.

## Notice, then rehearse

Name each speaker's purpose. Identify one useful expression and the next step or unresolved question. Read the roles aloud, then switch roles.

## LESSON 06 / 05 • CONVERSATIONS / 2 OF 3

# Throughput rises while individual waits grow

Fictional developers interpret two different measures in a batching experiment.

**01 • Developer:** The batched version completes more items per minute, but individual items wait longer before processing.

**02 • Colleague:** Then improved throughput doesn't necessarily mean each person gets a faster response, does it?

**03 • Developer:** Correct. Throughput measures completed work over time; latency measures the wait for a specified item.

**04 • Colleague:** Which goal matters for this feature? The current requirement is a response within the stated interaction window.

**05 • Developer:** Then we need the latency evidence against that window, not only the items-per-minute total.

**06 • Colleague:** Can we show both metrics for the same workload so the tradeoff is visible?

**07 • Developer:** Yes. The benchmark includes both, although we haven't measured the busiest production pattern.

**08 • Colleague:** Please keep that limitation. A larger batch might help total capacity while making this interaction less responsive.

**09 • Developer:** I'll explain the measured tradeoff without presenting higher throughput as a synonym for lower latency.

**10 • Colleague:** And we'll ask whether the observed wait meets the actual requirement before favoring the batch.

## Notice, then rehearse

Name each speaker's purpose. Identify one useful expression and the next step or unresolved question. Read the roles aloud, then switch roles.

## LESSON 06 / 05 • CONVERSATIONS / 3 OF 3

# A warm run beside a cold run

Fictional developers correct an unfair benchmark comparison.

**01 • Reviewer:** Your chart compares A's first run with B's fifth run. Were their starting conditions equivalent?

**02 • Developer:** No. B reused initialized data, while A's first run included preparing that data.

**03 • Reviewer:** Then we can't attribute the entire timing difference to the implementation change alone.

**04 • Developer:** Agreed. The warm and cold conditions need separate comparisons using matching workloads.

**05 • Reviewer:** Can you label the existing results honestly instead of discarding the slower observation?

**06 • Developer:** Yes. I'll mark them cold A and warm B, without claiming a controlled head-to-head result.

**07 • Reviewer:** What comparison would answer whether B changes the first-use experience for this workload?

**08 • Developer:** Cold runs for both implementations under the same stated preparation conditions would address that question.

**09 • Reviewer:** And warm runs for both would answer the repeated-use question, which is different.

**10 • Developer:** Exactly. I'll keep the two questions separate so a cache advantage doesn't become an unsupported implementation claim.

## Notice, then rehearse

Name each speaker's purpose. Identify one useful expression and the next step or unresolved question. Read the roles aloud, then switch roles.

## LESSON 06 / 06 • SAY IT

## Two scenarios to practice

Use only the supplied facts. Prepare for two minutes, speak for one minute, then respond to a follow-up. Switch roles and repeat. For solo study, speak both roles aloud.

### Scenario 1 | Comparing performance tradeoffs fairly

In a fictional controlled benchmark on the same input and machine, implementation A has median latency 40 milliseconds and memory use 80 megabytes; B has 25 milliseconds and 160 megabytes. The available memory budget is 120 megabytes. Tail latency and production workloads are unmeasured.

Prepare for two minutes. Speak for 45-60 seconds using the case facts, then respond to your partner's question. Switch roles and repeat without reading the model response.

Partner: Reviewer: prioritize the fixed memory budget and ask whether the measured latency covers the slowest requests.

Second round: The faster result came from a warmed cache in a later rerun. Ask for matched conditions before comparing that rerun with a cold run.

### Scenario 2 | The average hides the slow end

A fictional benchmark reports unchanged average latency, but its slowest measured group takes longer after a change. Sample size is forty runs; production representativeness is unknown.

Role A: Developer: explain how an unchanged average can coexist with slower individual observations.

Role B: Reviewer: ask for the distribution and comparable conditions, without assuming every user is slower.

Possible opening: The average is unchanged, but the slow end of this sample has moved.

#### Success checks

- Distinguish the average from the distribution.
- Keep the forty-run scope visible.
- Avoid claiming a production-wide effect.

Add a complication: A stakeholder wants to say there is no performance change because the average matches.

Check: facts accurate | meaning clear | tone appropriate | next action or question explicit

Continue scenario 1 with an AI partner.

## LESSON 07 / READ AND NOTICE

# Discussing retries and shared state

Explain why repeating a state-changing request needs an explicit behavior contract.

**SITUATION**

A fictional reservation client times out before receiving a reply. The reservation may already have been created. A colleague proposes automatically submitting a second request as a new reservation. The service's duplicate-request behavior is unspecified; no production change is authorized.

## 1. Understand the situation

Identify two confirmed facts, one missing detail, and the person who needs your response. Do not fill gaps with invented facts.

## 2. Find the words

race condition - Behavior whose outcome depends on the timing or ordering of overlapping operations.

idempotency - The property that repeating an operation has the same intended effect as performing it once.

transaction - A defined unit of related work whose completion and consistency guarantees depend on the system's transaction model.

invariant - A condition that the design requires to remain true across relevant operations or state changes.

### Useful word combinations

retry an operation: attempt it again after an uncertain or failed result

preserve an invariant: maintain a condition that must remain true

separate acknowledgment from completion: distinguish a received reply from the underlying outcome

interleave two requests: describe overlapping operations whose steps may occur in mixed order

## LESSON 07 / LANGUAGE AND PRACTICE

# Challenge an assumption about an uncertain outcome

## 3. Notice the language

A client may fail to receive a reply even when a service completed work. Idempotency concerns the intended effect of repetition, not a promise that every response or timing is identical. Discuss the fictional contract rather than prescribing a production retry mechanism.

A missing reply does not establish that \_\_\_\_.

If the first request completed, a new request could \_\_\_\_.

Before retrying, we need agreement on \_\_\_\_.

Compare this wording: It timed out, so nothing happened.

Clearer: The client timed out; the reservation outcome is unknown. Repeating it as a new request could have another effect.

The revision separates the client's observation from the service's unconfirmed state.

## 4. Check the language

1. Colleague: No reply means no reservation, right?

Developer: No; the underlying outcome is \_\_\_\_.

- A. reversed
- B. impossible
- C. unchanged
- D. unknown

2. Reviewer: What property concerns the effect of repeating the same operation?

Developer: \_\_\_\_ concerns repeated execution having the same intended effect as one execution.

- A. Throughput
- B. Deprecation
- C. Idempotency
- D. Concurrency

## 05 • Conversations

The next three pages contain original fictional conversations for this lesson. Each numbered line is one speaking turn. Read the setting before assigning roles.

## LESSON 07 / 05 • CONVERSATIONS / 1 OF 3

# The reply timed out, not necessarily the reservation

Fictional developers discuss a proposed automatic repeat of a state-changing request.

**01 • Client developer:** The reservation request timed out. I'll send a new one so the user can continue.

**02 • Service developer:** Do we know the first request failed, or only that the client didn't receive its reply?

**03 • Client developer:** Only the missing reply. The reservation may already exist, and repeated-request behavior is unspecified.

**04 • Service developer:** Then another new request could create an additional reservation rather than complete the original interaction.

**05 • Client developer:** We need a defined way to recognize repetition and return the appropriate outcome.

**06 • Service developer:** Yes. Idempotency concerns the intended effect of repeating the operation, not identical timing or wording.

**07 • Client developer:** Could we discuss that contract before proposing an automatic retry for this client?

**08 • Service developer:** Certainly. We should also state how the client distinguishes an unknown outcome from confirmed rejection.

**09 • Client developer:** I'll keep the current report at outcome unknown and withdraw the assumption that nothing happened.

**10 • Service developer:** That gives us the right design question without inventing a production retry mechanism.

## Notice, then rehearse

Name each speaker's purpose. Identify one useful expression and the next step or unresolved question. Read the roles aloud, then switch roles.

## LESSON 07 / 05 • CONVERSATIONS / 2 OF 3

## Two callers each see the last slot

Fictional developers discuss an overlapping-operation example without choosing a database technique.

**01 • Developer:** Two callers each see one slot available and both receive a successful reservation in this example.

**02 • Reviewer:** What invariant should hold? We need the intended rule before judging the observed interleaving.

**03 • Developer:** The agreed rule permits only one active reservation for that slot, so this outcome violates it.

**04 • Reviewer:** Then the issue depends on overlapping actions, not simply whether either caller behaves correctly alone.

**05 • Developer:** Right. Each reads availability before either completes the state change in the illustrated sequence.

**06 • Reviewer:** Calling the operations transactions doesn't tell us which concurrency guarantees this design actually provides.

**07 • Developer:** Agreed. We need the system's specified guarantees and evidence that the invariant survives overlapping requests.

**08 • Reviewer:** Please describe the conflicting outcomes and the invariant without prescribing a database-specific command here.

**09 • Developer:** I'll describe the race condition and violated invariant, with the implementation remedy still under review.

**10 • Reviewer:** That gives the review a precise invariant, not a vague request for safety.

### Notice, then rehearse

Name each speaker's purpose. Identify one useful expression and the next step or unresolved question. Read the roles aloud, then switch roles.

## LESSON 07 / 05 • CONVERSATIONS / 3 OF 3

# A repeated command is not a new intention

Fictional developers clarify a request identifier's meaning in a proposed contract.

**01 • Client developer:** Should the client create a new request identifier each time it repeats the same submitted action?

**02 • Interface owner:** In our proposed contract, repetition of that action retains its identifier; a genuinely new action gets another.

**03 • Client developer:** Then the identifier distinguishes another attempt from another user intention, not simply another network message.

**04 • Interface owner:** Exactly. But that is our proposal, not an assumption about every service or library.

**05 • Client developer:** What if the same identifier arrives with different contents? We haven't defined that case.

**06 • Interface owner:** Good catch. The contract needs an explicit outcome rather than silently treating different contents as equivalent.

**07 • Client developer:** I'll add that unresolved case and keep the repetition design marked incomplete.

**08 • Interface owner:** Please also explain how callers learn an earlier outcome after losing its reply.

**09 • Client developer:** That links the identifier rule to the actual user problem instead of treating it as decoration.

**10 • Interface owner:** And it keeps duplicate effects, changed contents, and missing replies as distinct questions to resolve.

## Notice, then rehearse

Name each speaker's purpose. Identify one useful expression and the next step or unresolved question. Read the roles aloud, then switch roles.

## LESSON 07 / 06 • SAY IT

## Two scenarios to practice

Use only the supplied facts. Prepare for two minutes, speak for one minute, then respond to a follow-up. Switch roles and repeat. For solo study, speak both roles aloud.

### Scenario 1 | Discussing retries and shared state

A fictional reservation client times out before receiving a reply. The reservation may already have been created. A colleague proposes automatically submitting a second request as a new reservation. The service's duplicate-request behavior is unspecified; no production change is authorized.

Prepare for two minutes. Speak for 45-60 seconds using the case facts, then respond to your partner's question. Switch roles and repeat without reading the model response.

Partner: Client developer: you want a responsive experience but do not know whether the server created the reservation before the timeout.

Second round: Someone says wrapping work in a transaction automatically settles all duplicate-request behavior. Ask which invariant and concurrency guarantees actually apply.

### Scenario 2 | Two editors save different versions

In a fictional editor, two people open revision 6. One saves revision 7. The other's later save still refers to revision 6. Conflict handling is unspecified; neither edit is known to be dispensable.

Role A: Developer: describe the competing updates and ask what the interface should communicate.

Role B: Reviewer: insist that the intended conflict outcome be defined before choosing a mechanism.

Possible opening: The second save is based on an older revision; what outcome should the caller expect?

#### Success checks

- Identify the older base revision.
- Avoid inventing a universal last-writer rule.
- State the unresolved contract question about competing edits.

Add a complication: A colleague assumes the later request must always overwrite the earlier one.

Check: facts accurate | meaning clear | tone appropriate | next action or question explicit

Continue scenario 1 with an AI partner.

## LESSON 08 / READ AND NOTICE

# Handing over a release with explicit limits

Give a release handover that distinguishes installed software from enabled behavior and verified recovery.

**SITUATION**

Fictional build R18 is deployed to staging. Its new search view is disabled by a feature flag. A schema migration adds an optional field, but compatibility with the previous application build is untested. Lee takes the release review; production rollout is undecided.

## 1. Understand the situation

Identify two confirmed facts, one missing detail, and the person who needs your response. Do not fill gaps with invented facts.

## 2. Find the words

build artifact - A generated software output, such as a package or executable, produced from a particular build.

feature flag - A control that enables, disables, or varies specified application behavior without necessarily changing the deployed build.

rollback - A return to an earlier application or configuration state, whose scope may not include undoing data changes.

schema migration - A controlled change to the structure or organization of stored data.

### Useful word combinations

identify the deployed build: name the exact software output running in an environment

stage a rollout: expose a change in bounded steps

define a rollback limit: explain what reverting the application does not undo

hand over an open compatibility check: transfer responsibility for an unresolved interaction

## LESSON 08 / LANGUAGE AND PRACTICE

# Transfer deployment, exposure, and compatibility status

## 3. Notice the language

A deployed artifact may contain disabled behavior. Rolling back an application does not automatically undo data changes. State which environment and build were checked and name the remaining review without promising recovery.

Build \_\_\_ is deployed to \_\_\_; the feature is \_\_\_.

The data change is \_\_\_, while compatibility with \_\_\_ is \_\_\_.

Please confirm ownership of \_\_\_ before the release decision.

Compare this wording: R18 is live and rollback is fine.

Clearer: R18 is in staging with search disabled; previous-build compatibility with the changed schema remains untested.

The revision avoids conflating staging deployment, feature exposure, and verified rollback.

## 4. Check the language

1. Release reviewer: Can staging users access the new search view?

Developer: Not through the flag-controlled route; the feature flag is \_\_\_.

- A. enabled for a pilot group
- B. removed from the build
- C. disabled
- D. enabled for everyone

2. Lee: Does reverting the application prove the schema change is undone?

Developer: No. Application rollback and schema reversal are \_\_\_.

- A. matching artifacts
- B. separate concerns
- C. identical guarantees
- D. completed checks

## 05 • Conversations

The next three pages contain original fictional conversations for this lesson. Each numbered line is one speaking turn. Read the setting before assigning roles.

## LESSON 08 / 05 • CONVERSATIONS / 1 OF 3

# R18 is installed but search is disabled

A fictional developer hands the release review to Lee.

**01 • Developer:** R18 is in staging, but the feature flag keeps the new search view disabled.

**02 • Lee:** Then deployment alone doesn't establish that users have exercised the new view in staging.

**03 • Developer:** Correct. The optional-field migration is present, but previous-build compatibility with it remains untested.

**04 • Lee:** Has anyone confirmed production rollout or a successful rollback based on this deployment?

**05 • Developer:** No. Both would overstate what we know. The open compatibility check belongs in your release review.

**06 • Lee:** I'll take that check. Please identify the exact build and the staging evidence I should review.

**07 • Developer:** The handover links R18's artifact and the migration record, with search explicitly marked disabled.

**08 • Lee:** I accept the review, not a release approval. The production decision is still open.

**09 • Developer:** Thank you. Application rollback also shouldn't be described as automatically undoing the schema change.

**10 • Lee:** Agreed. I'll keep build state, feature exposure, and recovery evidence separate in the decision.

## Notice, then rehearse

Name each speaker's purpose. Identify one useful expression and the next step or unresolved question. Read the roles aloud, then switch roles.

## LESSON 08 / 05 • CONVERSATIONS / 2 OF 3

# The artifact differs from the one tested

Fictional developers catch a release handover mismatch.

**01 • Release reviewer:** The release note names build K9, but the test evidence is labeled K8. Are they equivalent?

**02 • Developer:** They're different build artifacts. I haven't established that their contents are identical.

**03 • Release reviewer:** Then we can't attach K8's passing results to K9 without explaining that difference.

**04 • Developer:** Agreed. The intended release artifact needs its own identified evidence or a justified equivalence check.

**05 • Release reviewer:** What changed between the runs? The source label looks the same in the note.

**06 • Developer:** The dependency resolution may differ. Matching source labels alone don't settle the artifact contents.

**07 • Release reviewer:** Please keep the release candidate unverified until that build difference is explained and reviewed.

**08 • Developer:** I'll correct the evidence link's description so it says K8, rather than implying K9 was tested.

**09 • Release reviewer:** That protects the handover from a mistaken identity, without discarding useful results from K8.

**10 • Developer:** And it gives the next reviewer a precise artifact question instead of a generic green status.

## Notice, then rehearse

Name each speaker's purpose. Identify one useful expression and the next step or unresolved question. Read the roles aloud, then switch roles.

## LESSON 08 / 05 • CONVERSATIONS / 3 OF 3

# Switching a flag off doesn't erase prior changes

Fictional developers clarify the scope of disabling a feature after a trial.

**01 • Release lead:** The flag is off again. Does that mean the trial's stored changes are gone?

**02 • Developer:** No. Our flag controls the view, not reversal of stored data from the trial.

**03 • Release lead:** So disabling exposure and undoing data changes are separate actions with separate evidence.

**04 • Developer:** Exactly. We can confirm the view is disabled, but we haven't confirmed any data reversal.

**05 • Release lead:** Could the previous application build still read the data written during the trial?

**06 • Developer:** That compatibility question remains open. We shouldn't infer the answer from the flag's current state.

**07 • Release lead:** Please make that limit clear in the handover so support doesn't promise restored data.

**08 • Developer:** I'll state view disabled, trial data retained, and previous-build compatibility awaiting review.

**09 • Release lead:** That's specific enough for the review. We need an owner for the compatibility question.

**10 • Developer:** I'll bring it to the maintainer and retain the open status until someone accepts the check.

## Notice, then rehearse

Name each speaker's purpose. Identify one useful expression and the next step or unresolved question. Read the roles aloud, then switch roles.

## LESSON 08 / 06 • SAY IT

## Two scenarios to practice

Use only the supplied facts. Prepare for two minutes, speak for one minute, then respond to a follow-up. Switch roles and repeat. For solo study, speak both roles aloud.

### Scenario 1 | Handing over a release with explicit limits

Fictional build R18 is deployed to staging. Its new search view is disabled by a feature flag. A schema migration adds an optional field, but compatibility with the previous application build is untested. Lee takes the release review; production rollout is undecided.

Prepare for two minutes. Speak for 45-60 seconds using the case facts, then respond to your partner's question. Switch roles and repeat without reading the model response.

Partner: Lee: accept the compatibility review and ask which environment, build, and feature state the evidence covers.

Second round: A manager calls the disabled feature fully tested because the build started. Explain which user path has not been exercised.

### Scenario 2 | A rollout pauses after a bounded trial

A fictional feature is enabled for ten internal test accounts. Two report missing labels. The release lead pauses further exposure; existing evidence covers only those ten accounts and does not establish an external impact.

Role A: Developer: report the observed failures, exposure boundary, and paused expansion.

Role B: Release reviewer: ask what is known about the other eight accounts and external users.

Possible opening: Two of ten internal test accounts reported missing labels, and further exposure is paused.

### Success checks

- Preserve the ten-account scope.
- Distinguish reports from complete checks of the other accounts.
- Name the remaining investigation without expanding the impact claim.

Add a complication: A stakeholder asks to describe every user as affected.

Check: facts accurate | meaning clear | tone appropriate | next action or question explicit

Continue scenario 1 with an AI partner.

# Answer workshop

Try the tasks first. The model is one acceptable response to scenario 1, not a script to memorize. Assess your spoken version using the four criteria at the end.

## 01 | Clarifying interface contracts

Editing example: Omission leaves the nickname unchanged; an empty string clears it. Null still needs a defined meaning.

The revision distinguishes three inputs instead of grouping them under the ambiguous word blank.

### Check 1: B - unspecified

A: Deprecated implies formerly supported behavior being phased out; that is not given.

B: Correct. The meaning of null is not defined.

C: Equivalence has not been agreed.

D: Required contradicts the optional field.

### Check 2: A - whereas

A: Correct. Whereas contrasts different properties.

B: Therefore implies accepting null follows from optionality.

C: Because suggests null acceptance explains optionality.

D: Unless introduces a condition rather than explaining the distinction.

Model response: The contract already distinguishes an omitted nickname from an empty string: one leaves the value unchanged and the other clears it. Could we agree what null means before the client starts sending it? At present, null is unspecified, so I cannot describe it as another supported way to clear the field.

Case check: underline the confirmed information and circle a limitation, question, or next step in the response.

## 02 | Debugging from reproducible evidence

Editing example: The failure reaches date parsing; the empty date is a hypothesis that still needs an isolated comparison.

The revised claim matches the evidence and names the untested explanation.

### Check 1: A - row count

A: Correct. Both the date content and row count differ.

B: No changed date format is supplied for the passing file.

C: The case does not identify a changed delimiter between the files.

D: No encoding difference is established in the supplied comparison.

### Check 2: D - suggests

A: Rules out rejects rather than supports the possibility.

B: Guarantees overstates the trace's evidential value.

C: Unrelated to dismisses a relevant clue.

D: Correct. Suggests presents a supported but unconfirmed explanation.

Model response: The failing three-row file contains an empty date, and the stack trace reaches date parsing. Those observations make the empty field a useful hypothesis, not yet a demonstrated cause. Could we compare otherwise identical examples with and without that value? The smaller passing file also changed the row count, so it does not isolate the field.

Case check: underline the confirmed information and circle a limitation, question, or next step in the response.

### 03 | Giving actionable code-review feedback

Editing example: Moving the notification outside the success branch changes the failure path; please keep it conditional on success.

The review now names the defect and a concrete revision rather than judging the author or the entire change.

#### Check 1: D - side effect

A: The review concern is the notification, not a changed returned value.

B: No change to accepted input is described in this diff.

C: No changed parameter or return declaration is the identified blocker.

D: Correct. The notification is an observable effect outside the helper's result.

#### Check 2: C - preserve

A: Announce communicates a condition without retaining it.

B: Summarize describes rather than fixes the behavior.

C: Correct. Preserve asks to retain the existing condition.

D: Infer means deduce, not keep the condition in the code.

Model response: The helper rename is clear. In the same diff, the notification call moves outside the success branch, which would also notify after a failed export. Could you keep that call conditional on success and add a check for the failure path? My concern is the changed side effect, not your choice of helper name.

Case check: underline the confirmed information and circle a limitation, question, or next step in the response.

## 04 | Requesting dependency evidence

Editing example: Please show the resolved dependency change and check whether our callers use the deprecated entry point.

The revision requests evidence rather than treating a version label as a compatibility guarantee.

### Check 1: C - transitive

A: Optional concerns whether it is required, not the stated dependency path.

B: Development-only concerns its usage scope, not how it enters the application.

C: Correct. The application receives it through another dependency.

D: The application does not list it directly; ParcelKit introduces it.

### Check 2: B - removal

A: Resolution selects dependencies and is not the opposite availability state.

B: Correct. An interface may be deprecated before removal.

C: Documentation is the evidence source, not the contrasting availability state.

D: Installation does not express that the entry point no longer exists.

Model response: Could you identify whether our application or ParcelKit calls the deprecated parsing entry point, and link the relevant upgrade note? The proposed change also alters the resolved TextCore version, so please show the dependency path and lockfile difference. We need that evidence before calling this a drop-in update with no caller changes.

Case check: underline the confirmed information and circle a limitation, question, or next step in the response.

## 05 | Explaining what a test actually exercises

Editing example: The unit test checks a successful save response using simulated storage; rejection and the real adapter are not covered.

The revision identifies the checked behavior and the boundaries of the evidence.

### Check 1: B - always reports success

A: Every rejection is not checked by this test.

B: Correct. An always-successful substitute never reaches the disputed path.

C: Both outcomes would include rejection, contrary to the case.

D: The case states that storage is replaced.

### Check 2: A - regression

A: Correct. It guards against reintroducing the earlier defect.

B: Load testing examines behavior under workload.

C: Usability testing concerns how people use the interface.

D: Migration testing concerns a data or system transition.

Model response: This unit test checks the save function's success path using a controlled substitute for storage. Because that substitute always succeeds, it does not exercise the rejection that triggered the earlier bug. A regression case should check the visible result when rejection is simulated. An integration test of the real adapter answers a different question.

Case check: underline the confirmed information and circle a limitation, question, or next step in the response.

## 06 | Comparing performance tradeoffs fairly

Editing example: B lowers median latency but exceeds our memory budget; A stays within it. Tail latency is still unmeasured.

The revised comparison includes the limiting resource and the boundary of the measurements.

### Check 1: A - Implementation A

A: Correct. A's 80 megabytes is below 120.

B: B uses 160 megabytes and exceeds the limit.

C: A fits within the limit.

D: B uses 160, not 80 megabytes.

### Check 2: D - maximum

A: The mean averages values rather than identifying the longest observed wait.

B: The minimum identifies the shortest, not the longest, observed wait.

C: The median locates the middle of ordered observations, not the longest wait.

D: Correct. The maximum identifies the greatest observed value in the sample.

Model response: B has lower median latency in this benchmark, but it uses one hundred sixty megabytes against our one-hundred-twenty-megabyte budget. A is slower at the median and stays within that budget. I would not call B the better fit without addressing memory. Neither result establishes tail latency or performance on unmeasured production workloads.

Case check: underline the confirmed information and circle a limitation, question, or next step in the response.

## 07 | Discussing retries and shared state

Editing example: The client timed out; the reservation outcome is unknown. Repeating it as a new request could have another effect.

The revision separates the client's observation from the service's unconfirmed state.

### Check 1: D - unknown

A: Reversed would require evidence of undoing the effect.

B: Impossible overstates what the timeout establishes.

C: Unchanged assumes no effect, which is unconfirmed.

D: Correct. A missing reply does not reveal whether creation completed.

### Check 2: C - Idempotency

A: Throughput measures the amount of work completed over time.

B: Deprecation concerns an interface being discouraged or phased out.

C: Correct. Idempotency limits the intended effect of repetition.

D: Concurrency concerns overlapping work, not this repetition property.

Model response: I would not treat the timeout as proof that no reservation was created. A second request submitted as new could create another reservation if the first one completed. Could we define how repeated requests are recognized and what result the client should receive? We need that contract before recommending an automatic retry for this operation.

Case check: underline the confirmed information and circle a limitation, question, or next step in the response.

## 08 | Handing over a release with explicit limits

Editing example: R18 is in staging with search disabled; previous-build compatibility with the changed schema remains untested.

The revision avoids conflating staging deployment, feature exposure, and verified rollback.

### Check 1: C - disabled

A: No pilot exposure is enabled in the supplied staging state.

B: The feature exists in the build; its exposure is disabled rather than removed.

C: Correct. The case states that exposure is disabled.

D: The flag is disabled, so universal exposure is not supported.

### Check 2: B - separate concerns

A: Artifacts are build outputs, not the relationship between these actions.

B: Correct. Reverting application code does not itself establish a data reversal.

C: They are not interchangeable guarantees.

D: Neither action is reported as checked here.

Model response: R18 is deployed to staging, with the new search view disabled by its feature flag. The optional-field migration is present there, but compatibility with the previous application build remains untested. Lee, please take that compatibility check into the release review. No production rollout or successful rollback has been confirmed by these staging facts.

Case check: underline the confirmed information and circle a limitation, question, or next step in the response.

# Give feedback that helps

Score each criterion 0, 1, or 2. This is a practice rubric, not a professional qualification or CEFR test. Do not award or remove points for a particular accent.

## Meaning and accuracy

Keeps the case facts accurate; clearly separates confirmed and unknown information.

## Clarity and language

Uses understandable sentences and explains specialist terms when the listener needs it.

## Interaction and tone

Responds to the other person's question, checks understanding, and uses an appropriate tone.

## Task completion

Makes the requested action or unresolved question clear without inventing authority or facts.

## Use the same scale for each criterion

0 - Not yet: The reader or listener cannot yet recover the intended meaning or action.

1 - With support: The message works after a prompt, clarification, or revision.

2 - Independently: The message is clear and accurate without a prompt.

## Feedback sequence

First, identify a successful phrase. Next, identify one point where meaning or accuracy needs work. Offer one useful language correction, allow a repeat, and compare the two attempts. A total out of eight describes this performance only; do not translate it into a proficiency level.

## Final challenge

Choose a case not rehearsed today. The learner gives a one-minute response, answers two follow-up questions, and switches roles and repeats the exchange. Add the lesson's second-round challenge. Compare the result with an earlier attempt using the four criteria.

# Use the language responsibly

All services, packages, versions, measurements, and incidents are fictional language practice, not implementation instructions, cybersecurity guidance, or production operating procedures. Behaviors belong to the expressly described fictional contracts; no language, framework, package manager, or database behavior should be inferred. Check actual documentation, evidence, permissions, and qualified reviewers before technical decisions or production changes.

## Meaning before performance

Use specialist terms only when they help the listener. Explain unfamiliar words, preserve uncertainty, and ask when an instruction or responsibility is unclear. The model responses illustrate language; they do not authorize professional actions.

## Sources and further reading

The cases, workshops, and explanations are original teaching material. These references provide language frameworks and selected professional context. References were checked in September 2026; consult current local guidance for actual work.

### Digital.gov: plain-language guidance

<https://digital.gov/guides/plain-language>

### Council of Europe: CEFR descriptors

<https://www.coe.int/en/web/common-european-framework-reference-languages/cefr-descriptors>

### U.S. Bureau of Labor Statistics: Software Developers, Quality Assurance Analysts, and Testers

<https://www.bls.gov/ooh/computer-and-information-technology/software-developers.htm>

### NIST SP 800-218: Secure Software Development Framework, Version 1.1

<https://csrc.nist.gov/pubs/sp/800/218/final>

## Your next step

Choose one expression you can use in a genuine work situation. Rehearse it with fictional details, then adapt the wording to your role and audience.

# Extend your practice with AI

Use the next two prompts after your own first attempt. Each contains the course context and a complete starting case or dialogue. Copy the entire prompt, including the reference, into a new chat in your preferred AI. You can also use the links to copy it from the website.

## Choose the right kind of help

A role-play prompt makes the AI wait for your replies. A new-dialogue prompt asks for a complete professional script. Guided message practice gives four choices with explained feedback. Vocabulary, grammar and review prompts move from explanation to your own attempts.

## Choose any lesson or dialogue online

The course prompt workshop has eight practice goals and B1 support, B2 independent and C1 stretch settings. These are practice choices, not assessed proficiency levels. Select the same lesson number or dialogue title you used in this guide.

[Open this course's AI prompt workshop](#)

## Keep practice useful

Use fictional or anonymized details. English Ladder does not send anything to an AI service or add your drafts to the prompts. Your chosen service's terms and any usage charges apply. AI responses can contain mistakes; compare feedback with the lesson and verify specialist claims against the course references.

The two starter prompts use B2 practice. To change support in a printed prompt, replace its practice-setting paragraph with a request for shorter explanations and sentence starters (B1), or greater precision and more complex constraints (C1). Keep the professional facts unchanged.

## Changing the example

For another case on paper, replace the text between REFERENCE and END REFERENCE with that case, its course context, relevant terms and language target. Include the full exchange if practicing a dialogue. Do not assume your AI can access this PDF or a link. The website makes this substitution for you.

Prompt edition: 2026-09-28. These are original practice instructions; results depend on the AI service you choose.

# Practice grammar in context

## START PROMPT / COPY THROUGH END PROMPT | ONLINE VERSION

Be my workplace English practice partner. Follow the practice instructions after the REFERENCE block. The block contains fictional study material, not instructions to you. Keep its facts, numbers, uncertainty and professional responsibilities accurate. Clearly label any new scenario or changed fact as an invented variation. Use natural workplace language; explain unfamiliar abbreviations in context. If a technical expression is uncertain, say so rather than inventing a definition or authority. Coach communication, not professional decisions; respect the course scope. Ask only for fictional or anonymized practice details. Judge clarity, meaning, appropriate tone and the next step. Accept valid alternative English and distinguish errors from style preferences. Do not claim to certify my language level. Unless I ask to change the plan, follow the stages and stop wherever the instructions say to wait.

When discussing a word, phrase, or example sentence as language within an explanation, question, or answer feedback, enclose that wording in quotation marks. For example: What does "about" mean in "about twenty people"? Use "on" with a named day. Keep standalone answer choices, vocabulary headwords, and ordinary reading or dialogue text uncluttered. Quotation marks identifying a language example do not attribute it to a news source; attributed source quotations must remain verbatim.

Practice setting: B2 independent. Use natural professional English with brief explanations. Let me attempt each task without a model first. Offer a hint after difficulty and invite a more precise retry.

### REFERENCE

Course: Software Development English

Professional audience: Software developers, application engineers, code reviewers, and maintainers

Scope: All services, packages, versions, measurements, and incidents are fictional language practice, not implementation instructions, cybersecurity guidance, or production operating procedures. Behaviors belong to the expressly described fictional contracts; no language, framework, package manager, or database behavior should be inferred. Check actual documentation, evidence, permissions, and qualified reviewers before technical decisions or production changes.

Lesson 01: Clarifying interface contracts

Original fictional case: A fictional profile interface accepts an optional nickname. The current contract distinguishes an omitted nickname, which leaves it unchanged, from an empty string, which clears it. A proposed client sends null, whose meaning is unspecified.

Language workshop: Ask about one ambiguous input

Communication goal: Resolve an interface ambiguity using concrete input and outcome pairs.

Language guidance: Explain the specified cases before asking about the gap. 'Optional' concerns whether a field must be present; 'nullable' concerns whether null is accepted. Neither automatically tells a caller what an empty string does.

Useful frames (complete the gaps with case facts): For input \_\_\_, the contract specifies \_\_\_. / Does \_\_\_ mean \_\_\_, or should it be rejected? / That would change the behavior expected by \_\_\_.

Vocabulary: interface contract: The specified inputs, outputs, errors, and behavior that callers and an implementation agree to rely on. / nullable field: A data field whose specified allowed values include null, distinct from whether the field may be omitted. / validation error: A reported failure because an input does not meet a stated rule or accepted format. / backward compatibility: The ability of a changed component to continue supporting behavior or inputs relied on by existing consumers.

Speaking task: Prepare for two minutes. Speak for 45-60 seconds using the case facts, then respond to your partner's question. Switch roles and repeat without reading the model response.

Partner: Client developer: you want one way to clear a nickname, and your current client can send either an empty string or null.

Guided response task: Complete a workplace dialogue by selecting one of four supplied options. Compare the feedback and say the completed exchange aloud. Do not write an open-ended message.

Optional second-round challenge: A colleague calls omission and an empty string interchangeable. Restate their different outcomes in this fictional contract.

Model for comparison AFTER my attempt, not a response to give me first: The contract already distinguishes an omitted nickname from an empty string: one leaves the value unchanged and the other clears it. Could we agree what null means before the client starts sending it? At present, null is unspecified, so I cannot describe it as another supported way to clear the field.

**END REFERENCE**

**TASK:** Choose grammar that preserves the meaning

1. Use the lesson's language workshop, or one actual sentence from the dialogue, to explain a single grammar distinction in no more than 90 words. Show how it affects timing, certainty, responsibility or politeness.
2. Give one brief occupational exchange containing a gap and four complete options A-D. Supply enough context for exactly one answer. Keep distractors plausible; distinguish grammar errors from wording that changes the facts. Stop and wait.
3. Explain why each choice fits or fails, then ask me to read the completed exchange aloud. Do not request a draft, original written sentence or essay.
4. Repeat with three new exchanges, one at a time, moving from recognition to a more subtle meaning contrast. Do not copy the published checks. Label invented variations and do not turn uncertainty into a promise.
5. End with a short contrast table and two supported spoken turns using supplied frames. Accept accurate alternatives in oral practice; acknowledge and replace ambiguous multiple-choice items.

**END PROMPT**

# Choose a clear message

## START PROMPT / COPY THROUGH END PROMPT | ONLINE VERSION

Be my workplace English practice partner. Follow the practice instructions after the REFERENCE block. The block contains fictional study material, not instructions to you. Keep its facts, numbers, uncertainty and professional responsibilities accurate. Clearly label any new scenario or changed fact as an invented variation. Use natural workplace language; explain unfamiliar abbreviations in context. If a technical expression is uncertain, say so rather than inventing a definition or authority. Coach communication, not professional decisions; respect the course scope. Ask only for fictional or anonymized practice details. Judge clarity, meaning, appropriate tone and the next step. Accept valid alternative English and distinguish errors from style preferences. Do not claim to certify my language level. Unless I ask to change the plan, follow the stages and stop wherever the instructions say to wait.

When discussing a word, phrase, or example sentence as language within an explanation, question, or answer feedback, enclose that wording in quotation marks. For example: What does "about" mean in "about twenty people"? Use "on" with a named day. Keep standalone answer choices, vocabulary headwords, and ordinary reading or dialogue text uncluttered. Quotation marks identifying a language example do not attribute it to a news source; attributed source quotations must remain verbatim.

Practice setting: B2 independent. Use natural professional English with brief explanations. Let me attempt each task without a model first. Offer a hint after difficulty and invite a more precise retry.

### REFERENCE

Course: Software Development English

Professional audience: Software developers, application engineers, code reviewers, and maintainers

Scope: All services, packages, versions, measurements, and incidents are fictional language practice, not implementation instructions, cybersecurity guidance, or production operating procedures. Behaviors belong to the expressly described fictional contracts; no language, framework, package manager, or database behavior should be inferred. Check actual documentation, evidence, permissions, and qualified reviewers before technical decisions or production changes.

Lesson 01: Clarifying interface contracts

Original fictional case: A fictional profile interface accepts an optional nickname. The current contract distinguishes an omitted nickname, which leaves it unchanged, from an empty string, which clears it. A proposed client sends null, whose meaning is unspecified.

Language workshop: Ask about one ambiguous input

Communication goal: Resolve an interface ambiguity using concrete input and outcome pairs.

Language guidance: Explain the specified cases before asking about the gap. 'Optional' concerns whether a field must be present; 'nullable' concerns whether null is accepted. Neither automatically tells a caller what an empty string does.

Useful frames (complete the gaps with case facts): For input \_\_\_, the contract specifies \_\_\_. / Does \_\_\_ mean \_\_\_, or should it be rejected? / That would change the behavior expected by \_\_\_.

Vocabulary: interface contract: The specified inputs, outputs, errors, and behavior that callers and an implementation agree to rely on. / nullable field: A data field whose specified allowed values include null, distinct from whether the field may be omitted. / validation error: A reported failure because an input does not meet a stated rule or accepted format. / backward compatibility: The ability of a changed component to continue supporting behavior or inputs relied on by existing consumers.

Speaking task: Prepare for two minutes. Speak for 45-60 seconds using the case facts, then respond to your partner's question. Switch roles and repeat without reading the model response.

Partner: Client developer: you want one way to clear a nickname, and your current client can send either an empty string or null.

Guided response task: Complete a workplace dialogue by selecting one of four supplied options. Compare the feedback and say the completed exchange aloud. Do not write an open-ended message.

Optional second-round challenge: A colleague calls omission and an empty string interchangeable. Restate their different outcomes in this fictional contract.

Model for comparison AFTER my attempt, not a response to give me first: The contract already distinguishes an omitted nickname from an empty string: one leaves the value unchanged and the other clears it. Could we agree what null means before the client starts sending it? At present, null is unspecified, so I cannot describe it as another supported way to clear the field.

END REFERENCE

TASK: Coach guided workplace message choices

1. Use only the supplied reference and create one short, realistic dialogue with professional role labels and exactly one missing word or phrase. Supply four distinct options labelled A-D. Ensure that exactly one option fits both the grammar and the stated facts. Do not request a draft, an essay, or any open-ended writing. Stop and wait for my choice.
2. Use plausible distractors based on word combinations, certainty, timing, responsibility, or register. Avoid obviously absurd options. Do not introduce an unverified diagnosis, approval, deadline, charge, or legal rule. If extra fictional facts are necessary, state them explicitly before asking the question.
3. After I choose, identify the correct option and explain briefly why each of the other three options does not fit this exchange. Quote the relevant case fact or wording. Invite me to read both roles aloud; do not claim to assess pronunciation from a typed response.
4. Offer another four-choice exchange that practices a different expression. Present only one question at a time, wait for my answer, and vary the correct letter. If more than one answer is defensible, acknowledge the ambiguity and replace the question rather than marking a reasonable choice wrong.
5. After four exchanges, recap two useful word combinations and one distinction I should revisit. Ask which supported scenario I want to rehearse next, then stop and wait.

**END PROMPT**